

# Deep Learning

## Exercise 5: PyTorch

Room: **BIN-1-B.01**

Instructor: Manuel Günther

Email: [guenther@ifi.uzh.ch](mailto:guenther@ifi.uzh.ch)

Office: AND 2.54

Friday, March 20, 2026, 10:15 – 12:00

# Outline

- 1 PyTorch
- 2 Datatype `torch.tensor` and its Operations
- 3 Building a Network
- 4 Data
- 5 Network Training
- 6 Custom Gradient Implementation

# Local Installation

## Cuda Installation (optional, but desired)

- Install Cuda driver (if you have a Cuda-enabled GPU)
  - List of supported GPUs: <https://developer.nvidia.com/cuda-legacy-gpus>
  - Install driver for your OS <https://www.nvidia.com/Download>

## PyTorch Installation

- Install `pytorch` via pip (see <https://pytorch.org/get-started/locally>):  
`pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu118`
- Extensive documentation and tutorials:
  - Tutorials: <https://pytorch.org/tutorials>
  - Reference documentation: <https://pytorch.org/docs/stable>

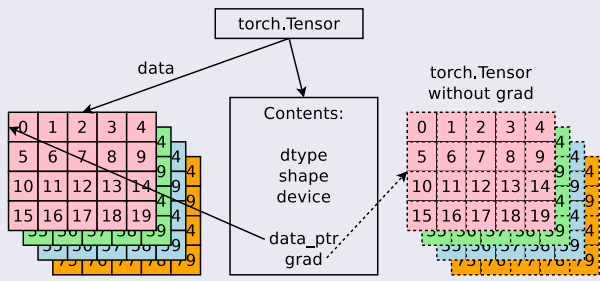
Everything is already installed in Google CoLab

# Datatype `torch.tensor`

## The `torch.tensor` Data Structure

- Multi-dimensional array (similar to `numpy.array`)
  - Has a specific datatype: `torch.float`, `torch.long`, ... and a `device`
  - Supports automatic differentiation: `backward` function and `grad` member

## Depiction of `torch.tensor`



# Datatype `torch.tensor`

## `torch.tensor` Creation

- Construct from data: `torch.tensor([[0,1],[2,3]])`
- Construct from `numpy`: `torch.as_tensor(data)`
- Initialize: `torch.empty(shape)`, `torch.zeros(shape)`, `torch.ones(shape)`
- Constructor parameters:
  - `dtype`; if not explicit from data, default: `float`
  - `requires_grad`; enable automatic differentiation, default: `False`

## `torch.tensor` Conversion to `numpy.array`

- Convert to `numpy.array` via: `x.detach().cpu().numpy()`
  - Detach from gradient via `detach()`, and put on CPU via `cpu()`
- Convert from **single-element** `torch.tensor` to float: `x.item()`

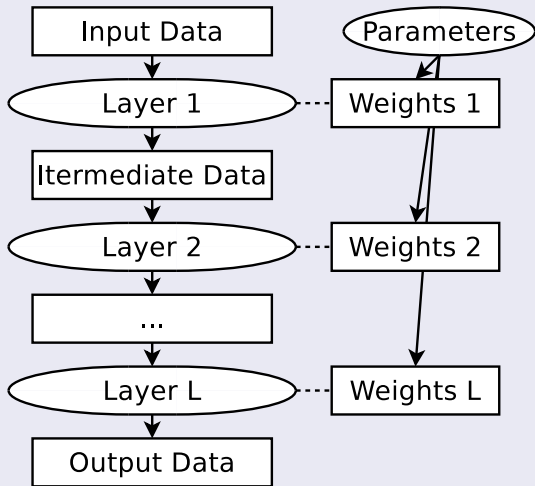
# Operations on `torch.tensor`

## Tensor Operations

- All operations consume **and produce** `torch.tensor` variables  
→ **Do not mix** `torch.tensor` and `numpy.array`!
- All math operations: `+`, `+=`, `*`, `*=`, `/`, `**`, ...
- Trigonometric: `torch.sin`, `torch.acos`, `torch.exp`, `torch.log`
- Reduction operations: `torch.max`, `torch.mean`, `torch.std`  
→ Select axis or axes to reduce via `dim` parameter (same as `axis` in `numpy`)
- Matrix operations: `torch.inner`, `torch.outer`, `torch.matmul`
- **All operations support automatic differentiation**
- Use `with torch.no_grad()`: block to avoid computing gradients  
→ Particularly useful for validation and testing

# Building a Network

## Typical Network Structure



## Definition of a Network

- Collection of functions/layers
- PyTorch-specific definition:  
Any function is a layer  
(different in `tensorflow`)
  - Functions with parameters,  
e.g.: fully-connected layers
  - Functions without parameters,  
e.g.: activation functions

## Building a Network

- Concatenation of layers
  - Defined in `forward(X)`

# Types of Layers

## Basic Layers

- Layers (typically) are derived from `torch.nn.Module`
  - Contain `parameters()` function to retrieve learnable parameters
  - `__call__(self, inputs)` calls `forward(self, inputs)`
- All layers work on batches of data

## Fully Connected Layer `torch.nn.Linear`

- Constructor parameters:
  - `in_features = D`; **no artificial bias neuron required**
  - `out_features = K`
  - `bias = True`
- Internal parameters: `weight` in  $\mathbb{R}^{K \times D}$  and `bias` in  $\mathbb{R}^K$



# Types of Layers

## 2D Conv. Layer `torch.nn.Conv2d`

- Constructor parameters:
  - `in_channels` =  $C$
  - `out_channels` =  $Q$
  - `kernel_size` =  $U$  or  $(U, V)$
  - `padding` = 0
  - `stride` = 1
  - `bias` = True
- Internal parameters:
  - `weight` in  $\mathbb{R}^{Q \times C \times U \times V}$
  - `bias` in  $\mathbb{R}^Q$

## Non-Learnable Layers

- Activation functions:  
`torch.nn.Sigmoid`, `torch.nn.Tanh`,  
`torch.nn.ReLU`, `torch.nn.Softmax`
- Pooling operations:  
`torch.nn.MaxPool2d`, `torch.nn.AvgPool2d`  
→ `kernel_size`, `stride`  
`torch.nn.AdaptiveMaxPool2d`
- Input flattening: `torch.nn.Flatten`
- Regularization layers:  
`torch.nn.Dropout`, `torch.nn.BatchNorm2d`

# Network Implementation

## Modules in PyTorch

- A module can be
  - A separate layer (e.g. Linear, ReLU, ...)
  - A block of layers (e.g. ResNet Block)
  - A complete network (e.g. LeNet, ResNet)
- ⇒ A network is a module
- Implemented as `torch.nn.Module`

## Connecting Layers

```
torch.nn.Sequential(layer1, layer2, ...)
```

## Defining a Custom Module

- Implemented as class (e.g. `Network`)
- Derive from `torch.nn.Module`:

```
class Network(torch.nn.Module):
```
- Two functions to implement
  - Constructor:

```
def __init__(self, ...)
```
  - Forward function:

```
def forward(self, x)
```

# Network Implementation

## The Constructor `def __init__(self, ...)`

- Call base class constructor (required, do not forget!):
  - `super(Network, self).__init__()`
- Instantiate all submodules:
  - `self.conv1 = torch.nn.Conv2d(...)`
  - `self.pooling = torch.nn.MaxPool2d(...)`
  - `self.activation = torch.nn.ReLU(...)`
  - `self.fc1 = torch.nn.Linear(...)`
- **If needed:** define variables as `Parameter`, similar to `torch.Tensor`
  - `self.my_param = torch.nn.Parameter(...)`
- All these are automatically extracted by `self.parameters()`

# Network Implementation

## The Forward Function `def forward(self, x)`

- Implement all processing steps of your network
  - `x = self.conv1(x)`
  - `x = self.pooling(x)`
  - `x = self.activation(x)`
- Can be grouped into (logical) blocks
  - `layer1 = self.activation(self.pooling(self.conv1(x)))`
- Can also use other non-parametric functions
  - `flattened = torch.flatten(layer1)`
- Can have multiple outputs (be aware of inplace functions)
  - `logits = self.fc1(flattened)`
  - `return logits, flattened`

# Datasets and Batches

## Datasets

- Many available in `torchvision.datasets`
  - `torchvision.datasets.MNIST`
  - `torchvision.datasets.ImageNet`
- Common interface:
  - `root`: directory of raw data
  - (`train: True` or `False`)
  - `download`: downloads data if not yet available in `root`
  - `transform`: preprocessing
- Return `PIL` images

## Transforms

- Prepares the input
  - Depends on original data
- Implemented in `torchvision.transforms`
  - `Resize((D, E))`: height, width
  - `Normalize(mean, std)`
  - `Lambda(callable)`: generic
  - `ToTensor`: `PIL` → `tensor`
- Combining several transforms:
  - `Compose((trans1, trans2))`
- Additionally: target transforms

# Datasets and Batches

## Data Loader

- Prepares batches of data
  - For both input and target
- `torch.utils.data.DataLoader`
  - `dataset`: see above
  - `batch_size = B`
  - `shuffle = False`:  
shuffle after each epoch
  - `num_workers = 0`:  
parallel execution on the CPU  
(might slow down processing)

## Example MNIST Dataset

```
# obtain datasets
transform = torchvision.transforms.ToTensor()
train_set = torchvision.datasets.MNIST(
    root="/temp/MNIST",
    train=True, download=True,
    transform=transform
)
test_set = torchvision.datasets.MNIST(
    root="/temp/MNIST",
    train=False, download=True,
    transform=transform
)

# loaders
train_loader = torch.utils.data.DataLoader(
    train_set, shuffle=True, batch_size=64
)
test_loader = torch.utils.data.DataLoader(
    test_set, shuffle=False, batch_size=100
)
```

# Custom Dataset Implementation

## Defining a Dataset

- Define dataset class (e.g. `MyDataset`)
- Derive from `torch.utils.data.Dataset`:  

```
class MyDataset(torch.utils.data.Dataset):
```
- Three functions to implement
  - Constructor: `def __init__(self, ...)`
  - Index function (sample at `index`): `def __getitem__(self, index)`
  - Length function (number of samples): `def __len__(self)`

# Dataset Implementation

## The Constructor `def __init__(self, transform=None, ...)`

- Call base class constructor (optional, to my knowledge):  
→ `super(MyDataset, self).__init__()`
- Instantiate all required variables:  
→ `self.data = ...`  
→ `self.targets = ...`  
→ `self.tansform = transform`

## The Length Function `def __len__(self)`

- Return the number of samples  $N$  in your dataset  
→ `return len(self.data)`



# Dataset Implementation

The Index Function `def __getitem__(self, index)`

- Typically returns `torch.tensors`
  - Required when further processed by `DataLoader`
  - Often defined by the `transform`
- Apply transform to data at index
  - `data = self.transform(self.data[index])`
- Compute target (integral value) at index
  - `target = self.targets[index]`
- Provide data and target items at this index
  - `return data, target`
  - **Can return tuple with arbitrary number of tensors**

# The Training Process

## Loss Functions

- Common interface: `forward(logits, targets) -> loss`
- `torch.nn.MSELoss`:  $\mathcal{J}^{L_2}$ 
  - Used mainly for regression tasks
- `torch.nn.BCEWithLogitsLoss`: **warning: not** `torch.nn.BCELoss`
  - Combines  $\mathcal{J}^{\text{BCE}}$  with  $\sigma$  activation (for easier gradient, see lecture)
  - Works directly on logit values; **no  $\sigma$  activation required in network**
  - Requires **target value** as `float` or `torch.float`
- `torch.nn.CrossEntropyLoss`:
  - Combines  $\mathcal{J}^{\text{CCE}}$  and SoftMax activation
  - Works directly on logit values; **no SoftMax activation required in network**
  - Requires **target class index**  $\tau$  as `int`, actually, as `torch.long`

# The Training Process

## Learning Strategy (Optimizer)

- Stochastic Gradient Descent `torch.optim.SGD` (or `torch.optim.Adam`)
  - `params`: All optimizable weights
  - `lr`: Learning rate  $\eta$
  - (only for SGD) `momentum`:  $\mu$  (default: 0)
- Function `step()`: Applies one gradient descent step
  - Requires `grad` variables to be populated for all `param` tensors
- `zero_grad()`: clears all `grad` variables from all `param` tensors

## Adaptive Learning Rate Schedulers

- Adapts learning rate of optimizer after each epoch
  - `torch.optim.lr_scheduler.StepLR`, `torch.optim.lr_scheduler.ReduceLROnPlateau`

# The Training Process

## Instantiation of all Required Elements

- ① Network: `network = torch.nn.Sequential(...)`
- ② Dataset(s): `dataset = torchvision.datasets.MNIST(...)`  
→ Often specific for training and validation set
- ③ Data loader(s): `loader = torch.utils.data.DataLoader(dataset, ...)`  
→ Often specific for training and validation set
- ④ Loss function: `loss_function = torch.nn.BCEWithLogitsLoss()`
- ⑤ Optimizer: `optimizer = torch.optim.SGD(network.parameters())`
- ⑥ Scheduler: `scheduler = torch.optim.lr_scheduler.StepLR(...)`  
→ Scheduler is optional

# The Training Process

## The Training Loop

- PyTorch training loops are very verbose
  - Many things to be implemented, but allows very flexible code
- Iterate over batches: `for inputs, targets in loader:`
  - 1 Forward batch through network: `logits = network(inputs)`
  - 2 Compute loss: `loss_value = loss_function(logits)`
    - Requirement: `loss_value` needs to be a scalar `torch.tensor`
  - 3 Compute gradients: `loss_value.backward()`
    - Populates `grad` variables for all `network.parameters()`
    - **Warning: Sums up with existing values**
  - 4 Perform gradient descent step: `optimizer.step`
  - 5 Clear gradients: `optimizer.zero_grad()`; **Do not forget!**
- Perform scheduler update: `scheduler.step(...)`

# The Training Loop

## Example Training Loop

```
# training loop
for epoch in range(epochs):
    for x,t in train_loader:
        # DO NOT FORGET:
        optimizer.zero_grad()
        z = network(x)
        J = loss(z, t)
        J.backward()
        optimizer.step()
        # collect training stats
        ...

    scheduler.step()
```

## Example Validation Loop

```
# validation loop
with torch.no_grad():
    correct = 0
    for x,t in val_loader:
        z = network(x)
        # optional: compute val loss
        J = loss(z, t)

        # compute val accuracy
        correct += torch.sum(
            torch.argmax(z, dim=1) == t
        ).item()
    acc = correct / len(val_set)
```

# Running on the GPU

## Preparation

- Test CUDA availability:  
`torch.cuda.is_available()`
- Select device "cpu" or "cuda":  
`device = torch.device("cuda")`
- Move everything to the device:  
`network.to(device)`  
`x.to(device)`  
`t.to(device)`

## Example Training Loop

```
# instantiate everything
device = torch.device("cuda")
network = network.to(device)
optimizer = torch.optim.SGD(
    params = network.parameters(),
    lr=0.01, momentum=0.9
)
# training epoch
for epoch in range(epochs):
    for x,t in train_loader:
        optimizer.zero_grad()
        z = network(x.to(device))
        J = loss(z, t.to(device))
        J.backward()
        optimizer.step()
```

# Running on the GPU

## Enabling GPU Processing on Google Colaboratory

- Google Colaboratory allows usage of Google GPU servers
- Open a notebook on Google Colaboratory
- Select Runtime → Change Runtime Type
- Select Acceleration GPU (does not work for TPU)
- Check that GPU is enabled:
  - `torch.cuda.is_available()` should return `True`



# Custom Gradient Implementation

## Processing of Derivatives in PyTorch

- Usually handled by automatic differentiation
- Defined for each operation in PyTorch
  - Enabled when using PyTorch functionality throughout
- Might not be optimal
  - For example  $\mathcal{J}^{\text{CCE}}$  on SoftMax has a simple derivative

## Implement your own Derivatives

- Implement a `torch.autograd.Function`
  - <https://pytorch.org/docs/stable/notes/extending.html#extending-torch-autograd>
- Define `forward` function with several inputs
- Provide partial derivative for each of the inputs in `backward`

# Custom Gradient Implementation

## Forward Pass

- Implement function  $\mathcal{Z} = f(\mathcal{A}, \mathcal{W})$
- Finally, a loss  $\mathcal{J}$  is computed from  $\mathcal{Z}$

## Backward Pass

- Goal: compute derivatives  $\frac{\partial \mathcal{J}}{\partial \mathcal{A}}$  and  $\frac{\partial \mathcal{J}}{\partial \mathcal{W}}$
- Provided to our function:  $\frac{\partial \mathcal{J}}{\partial \mathcal{Z}}$
- Compute:  $\frac{\partial \mathcal{J}}{\partial \mathcal{A}} = \frac{\partial \mathcal{J}}{\partial \mathcal{Z}} \frac{\partial \mathcal{Z}}{\partial \mathcal{A}}$  and  $\frac{\partial \mathcal{J}}{\partial \mathcal{W}} = \frac{\partial \mathcal{J}}{\partial \mathcal{Z}} \frac{\partial \mathcal{Z}}{\partial \mathcal{W}}$

# Custom Gradient Implementation

## The `forward` Function

- Is defined as static method via `@staticmethod` decorator  
→ Belongs to the class, not to the object

- Takes all parameters that your operation requires

```
@staticmethod
```

```
def forward(ctx, param1, param2, ...):
```

- Provides context information via `ctx`  
→ Can store required variables to the `backward` function

```
    ctx.save_for_backward(param1, param2)
```

- Returns the result of your operation

```
    result = operation(param1, param2)
```

```
    return result
```

# Custom Gradient Implementation

## The `backward` Function

- Also defined as static method via `@staticmethod` decorator
  - Has two parameters: context `ctx`, gradient w.r.t. output
- ```
@staticmethod  
def backward(ctx, grad):
```
- Extract stored information from context

```
    param1, param2 = ctx.saved_tensors
```
  - Return gradient for **each input of** `forward`
    - Need to be of same shape as input parameters; the applied Jacobian
    - Can be `None` if derivative for one parameter makes no sense

```
    derivative_for_param1 = grad * ...  
    return derivative_for_param1, None
```

# Example: CCE with SoftMax

## CCE Loss with One-Hot Vector

$$\mathcal{J}^{\text{CCE}} = -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O t_o^{[n]} \log y_o^{[n]} \quad \text{with} \quad \vec{y}^{[n]} = \text{SoftMax}(\vec{z}^{[n]})$$

## Partial Derivative of Loss

$$\frac{\partial \mathcal{J}^{\text{CCE}}}{\partial z_o^{[n]}} = \frac{\partial \mathcal{J}^{\text{CCE}}}{\partial \mathcal{J}^{[n]}} \cdot \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \quad \text{with} \quad \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} = y_o^{[n]} - t_o^{[n]}$$

## To be Implemented

- Forward pass to compute  $\mathcal{J}^{\text{CCE}}$  from a batch
- Backward pass to compute  $\frac{\partial \mathcal{J}^{\text{CCE}}}{\partial z_o^{[n]}}$  for a given  $\frac{\partial \mathcal{J}^{\text{CCE}}}{\partial \mathcal{J}^{[n]}}$

# Custom Gradient Implementation

## Example: CCE with SoftMax

```
class CCESoftMax(torch.autograd.Function):  
  
    # implement the forward propagation  
    @staticmethod  
    def forward(ctx, logits, targets):  
        # compute the log probabilities via log_softmax  
        log_y = torch.nn.functional.log_softmax(logits, dim=1)  
  
        # save required values for backward pass  
        ctx.save_for_backward(log_y, targets)  
  
        # compute average loss per batch, sum over classes  
        loss_value = - torch.sum(log_y * targets) / len(logits)  
        return loss_value
```

# Custom Gradient Implementation

## Example: CCE with SoftMax (continued)

```
# implement Jacobian
@staticmethod
def backward(ctx, dJ_dJn):
    # get results stored from forward pass
    log_y, targets = ctx.saved_tensors

    # compute derivative of loss w.r.t. the logits
    y = torch.exp(log_y)
    dJ_dz = dJ_dJn * (y-targets)

    # return the derivatives; None for derivative for the targets
    return dJ_dz, None

# set softmax as our loss function
loss_function = CCESoftMax.apply
```